UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS

GUÍA DE LABORATORIO N.° 04

Fundamentos de Redes Neuronales Artificiales: del Perceptrón a un Clasificador con Keras

I. Presentación de la sesión

Esta guía retoma el dataset climático de Huancayo construido en la Semana 2 (pronostico_huancayo.csv) y lo utiliza como base para introducir, de forma práctica, la construcción de una red neuronal artificial (perceptrón multicapa) con la librería Keras sobre TensorFlow. La progresión va de lo conceptual y sintético (una red neuronal mínima entrenada sobre datos artificiales) a lo aplicado (un clasificador binario que predice si un día será lluvioso, entrenado sobre datos reales de la región).

El objetivo no es dominar la teoría matemática del descenso de gradiente, sino comprender, mediante la experimentación directa, cómo se construye, entrena y evalúa una red neuronal densa, y cómo se prepara un dataset propio para ese fin.

Objetivos de aprendizaje

- Explicar, con sus propias palabras y apoyándose en el código ejecutado, qué hace un perceptrón y por qué una red necesita funciones de activación no lineales.
- Construir una red neuronal densa (MLP) con la API Sequential de Keras, usando capas Input y Dense.
- Entrenar y evaluar un modelo de clasificación binaria, interpretando las curvas de pérdida y precisión.
- Preparar un dataset propio para una tarea de clasificación: limpieza, codificación, escalado y partición train/test.
- Comparar el desempeño de una red neuronal frente a un modelo baseline (regresión logística) sobre el mismo problema.

- Aplicar buenas prácticas de ingeniería de software (control de versiones, pruebas automatizadas, documentación) al trabajo con modelos de machine learning.

PARTE 1: Preparación del entorno y del dataset

Paso 1.1 — Verificar y actualizar el entorno virtual del proyecto

Continuamos trabajando sobre el mismo repositorio (tecemer-lab) usado en semanas anteriores. Abre una terminal en la carpeta del proyecto y activa el entorno virtual existente.

```
# Windows (PowerShell)
cd tecemer-lab
.venv\Scripts\Activate.ps1

# macOS / Linux
# cd tecemer-lab
# source .venv/bin/activate
```

Con el entorno activado (deberías ver "(.venv)" al inicio de la línea de comandos), instala las librerías nuevas que se usarán esta semana: TensorFlow (incluye Keras), scikit-learn y Matplotlib.

```
# Windows (PowerShell)
.venv\Scripts\pip install tensorflow scikit-learn matplotlib

# macOS / Linux
# .venv/bin/pip install tensorflow scikit-learn matplotlib
```

¿Por qué puede tardar la instalación?

TensorFlow es una librería grande (varios cientos de MB). La primera instalación puede tomar varios minutos según tu conexión; es normal.

Si tu equipo tiene más de 8 años o muy poca RAM (menos de 4 GB libres), TensorFlow puede funcionar lento. Los modelos de esta guía son intencionalmente pequeños para que se entrenen en segundos incluso en equipos modestos.

Paso 1.2 — Crear la carpeta de trabajo de la semana

```
# Windows (PowerShell)
New-Item -ItemType Directory -Path semana04 -Force
cd semana04

# macOS / Linux
# mkdir -p semana04 && cd semana04
```

Paso 1.3 — Copiar el dataset de la Semana 2

Reutilizaremos el archivo pronostico_huancayo.csv generado en el Laboratorio 02. Cópialo desde la carpeta de esa semana hacia semana04/. Si ya no tienes el archivo (por ejemplo, cambiaste de equipo), vuelve a ejecutar clima.py y analisis.py del Laboratorio 02, o descarga tu copia entregada previamente.

```
# Windows (PowerShell) — ejecutar desde la raíz del proyecto
Copy-Item pronostico_huancayo.csv semana04\pronostico_huancayo.csv

# macOS / Linux
# cp pronostico_huancayo.csv semana04/pronostico_huancayo.csv
```

> ⚠ **Si tu CSV de la Semana 2 tiene columnas distintas**
>
> Esta guía asume que el archivo tiene, como mínimo, las columnas fecha, temp_max, temp_min y precipitacion (o una columna equivalente de lluvia/precipitación). Si tus nombres de columna difieren, ajusta el Paso 1.5 en consecuencia —revisa primero con df.columns en una consola de Python.

Paso 1.4 — Ampliar el historial climático

El archivo pronostico_huancayo.csv de la Semana 2 se generó consultando solo el pronóstico a corto plazo, es probable que tenga muy pocos días registrados. Con tan pocos datos, la partición train/test del Paso 1.6 puede fallar y el modelo no tendría suficiente información para aprender nada útil.

Para resolverlo, crearemos un script independiente, dentro de esta misma carpeta (semana04/), que consulta el endpoint HISTÓRICO de Open-Meteo y amplía el archivo pronostico_huancayo.csv que ya copiaste en el Paso 1.3.

Crea el archivo ampliar_dataset.py dentro de semana04/:

```
import datetime as dt

import pandas as pd
import requests

# Mismas coordenadas usadas en el Laboratorio 02 (Huancayo)
LATITUD, LONGITUD = -12.07, -75.21
DIAS_HISTORIAL = 90   # cuántos días hacia atrás se consultan
DIAS_REZAGO = 5       # el archivo histórico de Open-Meteo tiene algunos días de rezago

fecha_fin = dt.date.today() - dt.timedelta(days=DIAS_REZAGO)
fecha_inicio = fecha_fin - dt.timedelta(days=DIAS_HISTORIAL)

url = "https://archive-api.open-meteo.com/v1/archive"
parametros = {
    "latitude": LATITUD,
    "longitude": LONGITUD,
    "start_date": fecha_inicio.isoformat(),
    "end_date": fecha_fin.isoformat(),
    "daily": "temperature_2m_max,temperature_2m_min,precipitation_sum",
    "timezone": "auto",
}

try:
    respuesta = requests.get(url, params=parametros, timeout=15)
    respuesta.raise_for_status()
except requests.exceptions.RequestException as error:
    print(f"No se pudo consultar la API histórica de Open-Meteo: {error}")
    raise SystemExit(1)

datos = respuesta.json()["daily"]

df_historico = pd.DataFrame({
    "fecha": datos["time"],
```

```
    "temp_max": datos["temperature_2m_max"],
    "temp_min": datos["temperature_2m_min"],
    "precipitacion": datos["precipitation_sum"],
})

# Sobrescribe el CSV copiado en el Paso 1.3 con el historial ampliado
df_historico.to_csv("pronostico_huancayo.csv", index=False)

print(f"Rango consultado: {fecha_inicio} a {fecha_fin}")
print(f"Filas obtenidas: {len(df_historico)}")
print("\nArchivo pronostico_huancayo.csv actualizado con historial ampliado.")
```

> ⚠ **Sobre el rezago del endpoint histórico**
>
> archive-api.open-meteo.com no incluye los últimos días: por eso end_date se calcula restando DIAS_REZAGO (5 días) a la fecha actual, en vez de usar la fecha de hoy directamente. Si al ejecutar el script obtienes una respuesta vacía o un error 400, intenta aumentar DIAS_REZAGO a 7 o 10.

Ejecuta el script:

```
# Windows (PowerShell)
pip install requests   # si aún no lo tienes instalado en este venv
python ampliar_dataset.py


# macOS / Linux
# pip install requests
# python ampliar_dataset.py
```

Verifica que el dataset ampliado tenga tamaño y balance de clases suficientes antes de continuar. Puedes ejecutar esta verificación directamente en una consola de Python:

```
import pandas as pd


df = pd.read_csv("pronostico_huancayo.csv")
n_lluvioso = (df["precipitacion"] > 0).sum()
n_no_lluvioso = (df["precipitacion"] == 0).sum()


print(f"Total de filas: {len(df)}")
print(f"Días lluviosos: {n_lluvioso}  |  Días no lluviosos: {n_no_lluvioso}")


assert len(df) >= 30, "Muy pocas filas; aumenta DIAS_HISTORIAL en ampliar_dataset.py"
assert n_lluvioso >= 5 and n_no_lluvioso >= 5, "Alguna clase tiene muy pocos casos; amplía el rango de fechas"
print("Dataset ampliado validado correctamente.")
```

☑ Punto de control — Paso 1.4

El script debe imprimir el rango de fechas consultado y el número de filas obtenidas (idealmente 60-90).

La verificación debe pasar sin errores de assert: al menos 30 filas totales, y al menos 5 días de cada clase (lluvioso / no lluvioso).

Si algún assert falla, aumenta DIAS_HISTORIAL en ampliar_dataset.py (por ejemplo a 120 o 150) y vuelve a ejecutar el script antes de continuar al Paso 1.5.

Paso 1.5 — Crear el script preparar_dataset.py (parte 1: carga y exploración)

Dentro de semana04/, crea el archivo preparar_dataset.py con el siguiente contenido inicial.

```
import pandas as pd

# 1. Cargar el dataset construido en la Semana 2
df = pd.read_csv("pronostico_huancayo.csv")
```

```
print("Dimensiones del dataset:", df.shape)
print("\nColumnas disponibles:")
print(df.columns.tolist())
print("\nPrimeras filas:")
print(df.head())
print("\nValores nulos por columna:")
print(df.isnull().sum())
```

Ejecuta el script y observa la salida antes de continuar:

```
# Windows (PowerShell)
.venv\Scripts\python preparar_dataset.py

# Alternativamente Windows (PowerShell):
python preparar_dataset.py

# macOS / Linux
# .venv/bin/python preparar_dataset.py
```

Paso 1.6 — Ampliar preparar_dataset.py: construir la variable objetivo y las características

Agrega el siguiente bloque AL FINAL del mismo archivo preparar_dataset.py (no borres lo anterior). Construiremos una variable objetivo-binaria dia_lluvioso (1 si hubo precipitación registrada ese día, 0 si no) y un pequeño conjunto de características derivadas.

```
# 2. Construir la variable objetivo binaria: ¿fue un día lluvioso?
#    Ajusta el nombre de columna "precipitacion" si en tu CSV se llama distinto.
df["dia_lluvioso"] = (df["precipitacion"] > 0).astype(int)

# 3. Construir una característica derivada: amplitud térmica diaria
df["amplitud_termica"] = df["temp_max"] - df["temp_min"]

# 4. Seleccionar las características (X) y la variable objetivo (y)
```

```
columnas_features = ["temp_max", "temp_min", "amplitud_termica"]
X = df[columnas_features].copy()
y = df["dia_lluvioso"].copy()

print("\nDistribución de la variable objetivo (0 = no lluvioso, 1 = lluvioso):")
print(y.value_counts())
print("\nEstadísticas descriptivas de las características:")
print(X.describe())
```

¿Por qué estas tres características?

temp_max y temp_min capturan el rango térmico del día, que en climas de sierra como Huancayo suele correlacionarse con la nubosidad y la probabilidad de lluvia.

amplitud_termica (la diferencia entre ambas) es una característica derivada: los días despejados suelen tener mayor amplitud térmica que los días nublados/lluviosos. Es un ejemplo sencillo de ingeniería de características (feature engineering).

Paso 1.7 — Ampliar preparar_dataset.py: limpieza, escalado y partición train/test

Agrega el siguiente bloque al final del archivo. Aquí se maneja cualquier valor nulo remanente, se estandarizan las características numéricas y se separan los datos en conjunto de entrenamiento y de prueba.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import numpy as np

# 5. Manejo de valores nulos: eliminar filas incompletas (dataset pequeño y ya validado en Sem. 2)
antes = len(X)
datos_completos = pd.concat([X, y], axis=1).dropna()
X = datos_completos[columnas_features]
y = datos_completos["dia_lluvioso"]
print(f"\nFilas eliminadas por valores nulos: {antes - len(X)}")

# 6. Partición train/test (80% / 20%), estratificada para preservar la proporción de clases
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
```

```
)

# 7. Escalado: las redes neuronales entrenan mejor con features en rangos similares
escalador = StandardScaler()
X_train_esc = escalador.fit_transform(X_train)
X_test_esc = escalador.transform(X_test)

print(f"\nTamaño de entrenamiento: {X_train_esc.shape}")
print(f"Tamaño de prueba: {X_test_esc.shape}")

# 8. Guardar los arreglos preparados para reutilizarlos en las siguientes partes
np.savez(
    "dataset_preparado.npz",
    X_train=X_train_esc, X_test=X_test_esc,
    y_train=y_train.values, y_test=y_test.values,
)
print("\nArchivo dataset_preparado.npz guardado correctamente.")
```

✅ **Punto de control — Parte 1**

Al ejecutar preparar_dataset.py debes ver, sin errores: las dimensiones del dataset original, la distribución de dia_lluvioso, las estadísticas de las features, el número de filas eliminadas (probablemente 0 si el dataset de la Semana 2 ya estaba limpio), los tamaños de train/test, y el mensaje final de guardado de dataset_preparado.npz.

Si la distribución de dia_lluvioso está muy desbalanceada (por ejemplo, 95% / 5%), coméntalo con el docente: puede requerir ajustes posteriores en la evaluación del modelo.

PARTE 2: Primer MLP con Keras sobre datos sintéticos

Antes de trabajar con el dataset real, construiremos una red neuronal sobre datos sintéticos generados con scikit-learn. Esto permite enfocarse en la arquitectura y el flujo de entrenamiento de Keras sin la complejidad adicional de un dataset real, y sirve como plantilla reutilizable para la Parte 3.

Paso 2.1 — Crear perceptron_sintetico.py: generar datos de ejemplo

```
from sklearn.datasets import make_classification
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Generar un dataset sintético de clasificación binaria
X, y = make_classification(
    n_samples=500,
    n_features=4,
    n_informative=3,
    n_redundant=0,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

escalador = StandardScaler()
X_train = escalador.fit_transform(X_train)
X_test = escalador.transform(X_test)

print("X_train:", X_train.shape, " X_test:", X_test.shape)
```

Ejecuta y confirma que no hay errores antes de continuar:

```
.venv\Scripts\python perceptron_sintetico.py    # Windows
# .venv/bin/python perceptron_sintetico.py    # macOS / Linux
```

Paso 2.2 — Ampliar perceptron_sintetico.py: definir la arquitectura del MLP

Agrega al final del mismo archivo la definición del modelo usando la API Sequential de Keras.

```
from tensorflow import keras
from tensorflow.keras import layers

# 2. Definir la arquitectura: perceptrón multicapa (MLP)
modelo = keras.Sequential([
```

```
    layers.Input(shape=(4,)),          # 4 características de entrada
    layers.Dense(8, activation="relu"),  # capa oculta 1
    layers.Dense(4, activation="relu"),  # capa oculta 2
    layers.Dense(1, activation="sigmoid"),  # salida: probabilidad de clase 1
])

modelo.summary()
```

Leyendo el resumen del modelo (model.summary())

Cada fila Dense muestra su forma de salida y su número de parámetros (pesos + sesgos). Por ejemplo, la primera capa Dense(8) tiene 4×8 pesos + 8 sesgos = 40 parámetros.

La capa final usa sigmoid porque produce un valor entre 0 y 1, interpretable como la probabilidad de que la instancia pertenezca a la clase 1 — ideal para clasificación binaria.

Las capas ocultas usan relu porque es computacionalmente eficiente y evita el problema de gradientes que se desvanecen, muy común con sigmoid en redes más profundas.

Paso 2.3 — Ampliar perceptron_sintetico.py: compilar el modelo

```
# 3. Compilar: definir optimizador, función de pérdida y métricas
modelo.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"],
)
```

¿Qué hace compile()?

optimizer="adam": el algoritmo que ajusta los pesos de la red en cada paso de entrenamiento (una variante eficiente del descenso de gradiente).

loss="binary_crossentropy": la función de pérdida adecuada para clasificación binaria; mide qué tan lejos están las probabilidades predichas de las etiquetas reales (0 o 1).

metrics=["accuracy"]: métrica adicional que se reporta durante el entrenamiento (no se usa para ajustar los pesos, solo para monitoreo).

Paso 2.4 — Ampliar perceptron_sintetico.py: entrenar y graficar

```
import matplotlib
```

```
matplotlib.use("Agg")   # backend sin ventana gráfica, guarda directo a archivo
import matplotlib.pyplot as plt

# 4. Entrenar el modelo
historial = modelo.fit(
    X_train, y_train,
    epochs=30,
    batch_size=16,
    validation_split=0.2,
    verbose=1,
)

# 5. Graficar la evolución de la pérdida y la precisión
fig, ejes = plt.subplots(1, 2, figsize=(10, 4))

ejes[0].plot(historial.history["loss"], label="entrenamiento")
ejes[0].plot(historial.history["val_loss"], label="validación")
ejes[0].set_title("Pérdida (loss)")
ejes[0].set_xlabel("Época")
ejes[0].legend()

ejes[1].plot(historial.history["accuracy"], label="entrenamiento")
ejes[1].plot(historial.history["val_accuracy"], label="validación")
ejes[1].set_title("Precisión (accuracy)")
ejes[1].set_xlabel("Época")
ejes[1].legend()

plt.tight_layout()
plt.savefig("curvas_entrenamiento_sintetico.png", dpi=100)
print("\nGráfico guardado en curvas_entrenamiento_sintetico.png")
```

⚠ **Sobre epochs, batch_size y validation_split**

epochs=30: el modelo recorre el conjunto de entrenamiento completo 30 veces. Con datos sintéticos y una red pequeña, 30 épocas son suficientes y rápidas (segundos).

batch_size=16: los pesos se actualizan tras ver grupos ("lotes") de 16 ejemplos, en vez de uno por uno o todos a la vez. Es un equilibrio estándar entre velocidad y estabilidad.

validation_split=0.2: Keras aparta automáticamente el 20% de X_train (no del test) para medir el desempeño en cada época sin usarlo para ajustar pesos — permite detectar sobreajuste (overfitting) temprano.

Paso 2.5 — Ampliar perceptron_sintetico.py: evaluar en el conjunto de prueba

```
# 6. Evaluar el modelo en datos nunca vistos (conjunto de prueba)
perdida_test, precision_test = modelo.evaluate(X_test, y_test, verbose=0)
print(f"\nPérdida en test: {perdida_test:.4f}")
print(f"Precisión en test: {precision_test:.4f}")
```

Punto de control — Parte 2

model.summary() debe mostrar 3 capas Dense con un total de parámetros mayor a cero.

El entrenamiento debe completar las 30 épocas sin errores, mostrando loss decreciente en la mayoría de las épocas.

Debe generarse el archivo curvas_entrenamiento_sintetico.png en la carpeta semana04/.

La precisión en test debería ubicarse, típicamente, entre 0.75 y 0.95 (dataset sintético, sujeto a variación por la semilla aleatoria).

PARTE 3: Clasificador de lluvia sobre datos reales

Aplicamos el mismo flujo de trabajo de la Parte 2, ahora sobre el dataset real preparado en la Parte 1 (dataset_preparado.npz), comparamos contra un modelo baseline simple, guardamos el modelo entrenado y construimos un pequeño script de inferencia.

Paso 3.1 — Crear clasificador_lluvia.py: cargar los datos preparados

```
import numpy as np

# 1. Cargar los arreglos guardados en la Parte 1
datos = np.load("dataset_preparado.npz")
X_train, X_test = datos["X_train"], datos["X_test"]
y_train, y_test = datos["y_train"], datos["y_test"]

print("X_train:", X_train.shape, " X_test:", X_test.shape)
print("Proporción de días lluviosos en train:", y_train.mean().round(3))
```

Paso 3.2 — Ampliar clasificador_lluvia.py: modelo baseline con regresión logística

Antes de usar una red neuronal, es una buena práctica establecer un modelo baseline simple. Si el MLP no supera a un modelo mucho más sencillo, probablemente no vale la pena su complejidad adicional.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 2. Baseline: regresión logística
baseline = LogisticRegression()
baseline.fit(X_train, y_train)
pred_baseline = baseline.predict(X_test)
precision_baseline = accuracy_score(y_test, pred_baseline)
print(f"\nPrecisión del baseline (regresión logística): {precision_baseline:.4f}")
```

Paso 3.3 — Ampliar clasificador_lluvia.py: construir y entrenar el MLP

```
from tensorflow import keras
from tensorflow.keras import layers
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

# 3. Definir, compilar y entrenar la red neuronal
modelo = keras.Sequential([
    layers.Input(shape=(X_train.shape[1],)),
    layers.Dense(8, activation="relu"),
    layers.Dense(4, activation="relu"),
    layers.Dense(1, activation="sigmoid"),
])

modelo.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

historial = modelo.fit(
    X_train, y_train,
    epochs=40,
```

```
    batch_size=8,
    validation_split=0.2,
    verbose=1,
)

perdida_test, precision_test = modelo.evaluate(X_test, y_test, verbose=0)
print(f"\nPrecisión del MLP en test: {precision_test:.4f}")
print(f"Precisión del baseline en test: {precision_baseline:.4f}")

plt.figure(figsize=(6, 4))
plt.plot(historial.history["loss"], label="entrenamiento")
plt.plot(historial.history["val_loss"], label="validación")
plt.title("Pérdida — clasificador de lluvia (Huancayo)")
plt.xlabel("Época")
plt.legend()
plt.tight_layout()
plt.savefig("curvas_entrenamiento_lluvia.png", dpi=100)
print("\nGráfico guardado en curvas_entrenamiento_lluvia.png")
```

> ⚠️ **Sobre la comparación MLP vs. baseline**
>
> Con un dataset pequeño (como el de una sola estación meteorológica y pocos meses de datos), es normal que el MLP no supere ampliamente al baseline, o incluso rinda igual o algo menos. Esto NO es un error: ilustra que un modelo más complejo no siempre es mejor, especialmente con poca cantidad de datos.
>
> Lo importante en este paso es el proceso —construir, entrenar, evaluar y comparar de forma rigurosa— no alcanzar una precisión perfecta.

Paso 3.4 — Ampliar clasificador_lluvia.py: guardar el modelo entrenado

```
# 4. Guardar el modelo entrenado para reutilizarlo sin reentrenar
modelo.save("modelo_lluvia.keras")
print("\nModelo guardado en modelo_lluvia.keras")
```

Paso 3.5 — Crear predecir.py: script de inferencia

Crea un archivo NUEVO llamado predecir.py (distinto de clasificador_lluvia.py) que cargue el modelo guardado y prediga sobre nuevas observaciones hipotéticas.

```
import numpy as np
from tensorflow import keras
 # 1. Cargar el modelo entrenado en el Paso 3.4
modelo = keras.models.load_model("modelo_lluvia.keras")
 # 2. Definir observaciones nuevas: [temp_max, temp_min, amplitud_termica]
#    IMPORTANTE: deben escalarse con el MISMO escalador usado en el entrenamiento.
#    Para esta demostración usamos valores ya aproximados a la escala estandarizada;
#    en un flujo de producción real, se guardaría también el objeto StandardScaler (ver Nota).
observaciones_nuevas = np.array([
    [0.5, 0.3, 0.1],    # ejemplo: día con amplitud térmica moderada
    [-1.2, -0.8, -0.4], # ejemplo: día con temperaturas bajas y poca amplitud
])

# 3. Predecir
probabilidades = modelo.predict(observaciones_nuevas)
predicciones = (probabilidades > 0.5).astype(int)

for i, (prob, pred) in enumerate(zip(probabilidades, predicciones)):
    etiqueta = "lluvioso" if pred[0] == 1 else "no lluvioso"
    print(f"Observación {i+1}: probabilidad={prob[0]:.3f} -> predicción: día {etiqueta}")
```

Nota sobre el escalador en producción

En un proyecto real, el objeto StandardScaler ajustado en el Paso 1.6 debe guardarse (por ejemplo, con joblib.dump) y cargarse junto al modelo, para aplicar exactamente la misma transformación a cualquier dato nuevo antes de predecir. En esta guía se simplifica el ejemplo para enfocarse en el flujo de carga y predicción del modelo; se deja como reto opcional.

Paso 3.6 — Prueba automatizada: verificar la función de preparación de datos

Como en semanas anteriores, cerramos el laboratorio con una prueba automatizada. Crea la carpeta tests/ y el archivo tests/test_preparacion.py. Primero, refactoriza brevemente preparar_dataset.py extrayendo la lógica de la variable objetivo a una función reutilizable; agrega esta función AL INICIO de preparar_dataset.py (antes del bloque "1. Cargar el dataset"):

```
def calcular_dia_lluvioso(precipitacion):
```

```
    """Devuelve 1 si hubo precipitación registrada, 0 en caso contrario.

    Args:
        precipitacion: valor numérico de precipitación (mm).

    Returns:
        int: 1 si precipitacion > 0, 0 en caso contrario.
    """
    return int(precipitacion > 0)
```

Y reemplaza la línea del Paso 1.6 que crea la columna dia_lluvioso por esta versión que reutiliza la función:

```
# Reemplazar:
# df["dia_lluvioso"] = (df["precipitacion"] > 0).astype(int)
# Por:
df["dia_lluvioso"] = df["precipitacion"].apply(calcular_dia_lluvioso)
```

Ahora crea tests/test_preparacion.py:

```
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent))
from preparar_dataset import calcular_dia_lluvioso


def test_dia_con_lluvia_devuelve_uno():
    assert calcular_dia_lluvioso(2.5) == 1


def test_dia_sin_lluvia_devuelve_cero():
    assert calcular_dia_lluvioso(0.0) == 0


def test_precipitacion_negativa_se_trata_como_sin_lluvia():
    # Caso borde: por consistencia de datos, no deberían existir valores
```

```
    # negativos, pero la función debe comportarse de forma predecible.
    assert calcular_dia_lluvioso(-1.0) == 0
```

> ⚠️ **Efecto secundario al importar preparar_dataset.py**
>
> Al importar preparar_dataset en la prueba, Python ejecuta TODO el script (incluida la carga del CSV y el entrenamiento... en este caso solo hasta el guardado del .npz). Esto funcionará porque pronostico_huancayo.csv está en la misma carpeta, pero es una limitación de diseño real: en un proyecto profesional, la lógica reutilizable se separaría en un módulo sin efectos secundarios al importarse. Coméntalo en tu README como una mejora futura identificada conscientemente.

```
# Windows (PowerShell)
.venv\Scripts\python -m pytest tests/ -v

#Alternativamente:
python -m pytest tests/ -v

# macOS / Linux
# .venv/bin/python -m pytest tests/ -v
```

Paso 3.7 — Documentar y subir los cambios a GitHub

Crea o actualiza el archivo README.md de semana04/ con una breve descripción del laboratorio, y confirma los cambios siguiendo el flujo de Conventional Commits usado en semanas anteriores.

```
git checkout -b feature/red-neuronal-lluvia
git add semana04/
git commit -m "feat: clasificador de lluvia con MLP (Keras) sobre dataset de Huancayo"
git push -u origin feature/red-neuronal-lluvia
```

Luego, abre una Pull Request en GitHub describiendo brevemente el modelo construido y el resultado obtenido frente al baseline, tal como se practicó en el Laboratorio 03.

> ✅ **Punto de control — Parte 3**
>
> clasificador_lluvia.py debe ejecutarse sin errores e imprimir la precisión del baseline y del MLP. Deben existir los archivos modelo_lluvia.keras y curvas_entrenamiento_lluvia.png en semana04/.

predecir.py debe ejecutarse cargando el modelo guardado (sin reentrenar) e imprimir una predicción por cada observación de ejemplo.

Las 3 pruebas de tests/test_preparacion.py deben pasar (3 passed).

Los cambios deben estar subidos a una rama feature/ en GitHub con una Pull Request abierta.

II. Entregables

1. Repositorio en GitHub con la carpeta semana04/ que incluye: preparar_dataset.py, perceptron_sintetico.py, clasificador_lluvia.py, predecir.py, tests/test_preparacion.py, modelo_lluvia.keras, ambos gráficos .png y README.md.
2. Pull Request abierta (o fusionada) desde la rama feature/red-neuronal-lluvia con una descripción breve del trabajo realizado.
3. Captura de pantalla o registro de la ejecución de pytest tests/ -v mostrando las 3 pruebas en verde.

III. Anexo — Estructura final de archivos

```
tecemer-lab/
└── semana04/
    ├── pronostico_huancayo.csv
    ├── preparar_dataset.py
    ├── dataset_preparado.npz
    ├── perceptron_sintetico.py
    ├── curvas_entrenamiento_sintetico.png
    ├── clasificador_lluvia.py
    ├── curvas_entrenamiento_lluvia.png
    ├── modelo_lluvia.keras
    ├── predecir.py
    ├── README.md
    └── tests/
        └── test_preparacion.py
```